

ADDON (MÓDULO) ODOO

Odoo.sh es la plataforma oficial de Odoo para alojar proyectos en la nube y desplegar desarrollos propios. Nosotros vamos a trabajar con Odoo instalado en local, pero la estructura de los addons es exactamente la misma.

La siguiente tabla resume las formas de trabajar con Odoo:

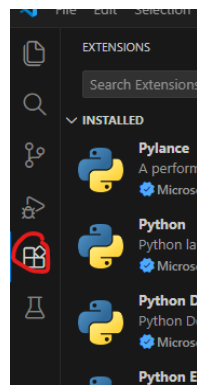
Odoo Online	Odoo en la nube, listo para usar. Todo gestionado por Odoo; no puedes tocar el código del sistema.	Empresas que quieren usar Odoo sin instalar nada ni programar.
Odoo On-Premise (local)	Odoo instalado en un PC o servidor propio. Se puede ver y modificar todo el código.	Aprendizaje, desarrollo de addons, pruebas y PYMEs que quieren control total.
Odoo.sh	Plataforma oficial de Odoo en la nube para desarrollo y despliegue de proyectos. Permite crear repositorios y manejar entornos de desarrollo y producción. Es de pago	Empresas que desarrollan addons propios y quieren un entorno profesional para desplegarlos.

[Este link](#) es la entrada al manual de Odoo para crear un addon (aunque es para [odoo.sh](#), sirve igual para Odoo onprem).

OBS! La versión 19 de ODOO es mucho más estricta al interpretar la sintaxis xml. Tenerlo en cuenta porque hay ejemplos en la red que funcionan en versiones anteriores de ODOO pero no con la última.

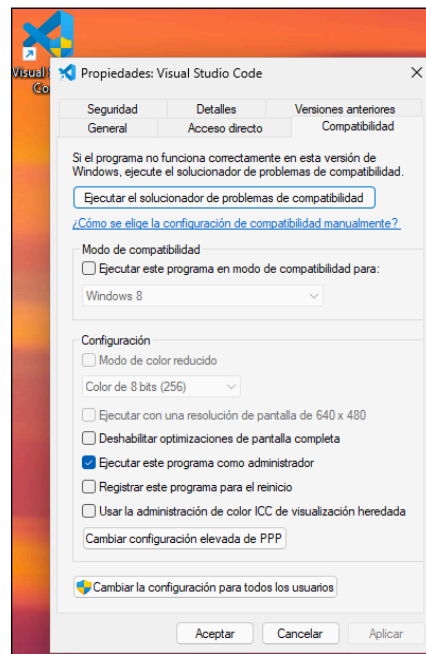
PREPARACIÓN ENTORNO DE TRABAJO

Vamos a utilizar VS Code:
<https://code.visualstudio.com/download>
con las extensiones: python y xml (redhat).



Seguramente necesitéis ejecutar VS code con permisos de administrador. Yo he creado un acceso directo que lo abre como administrador.

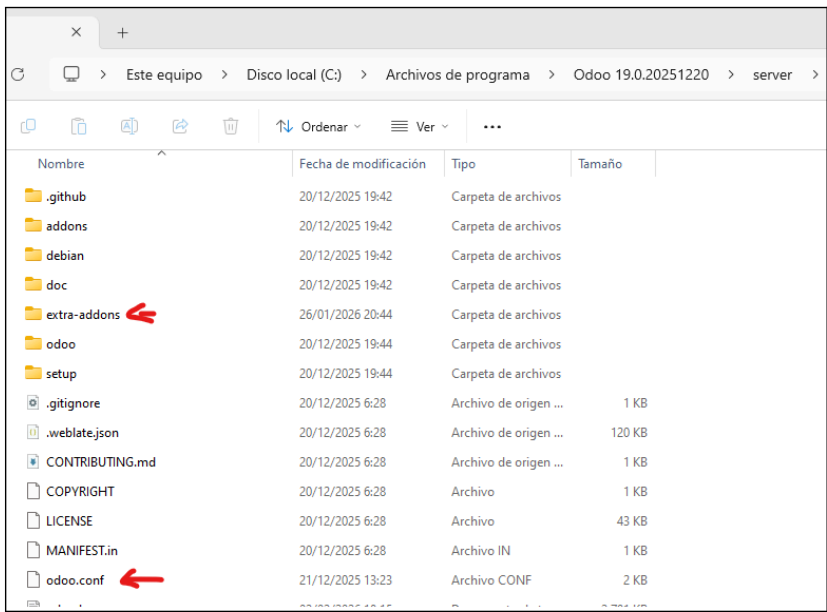
- Crear el acceso directo
- Entrar en propiedades, compatibilidad
- Marcar como en la figura



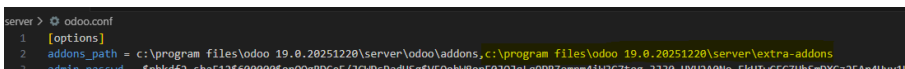
Esta figura muestra la instalación de Odoo.

He creado la carpeta **extra-addons** que contiene los addons (o módulos) de usuario.

Es necesario añadir la ruta en **addons_path** del fichero **odoo.conf**.



... tal como se muestra en la figura.



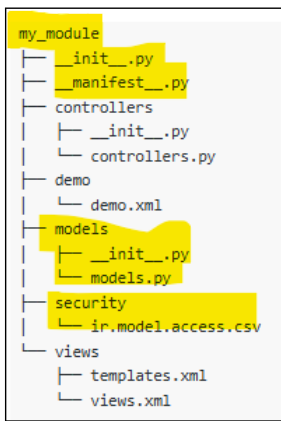
La figura muestra la estructura típica de un addon llamado **my_module**.

He marcado en amarillo las partes que tienen que existir. El resto depende de cómo el desarrollador quiere estructurarlo.

Yo los que he creado los he ido creando 'a pelo' con la ayuda de chatGPT, pero seg. el link del principio, podéis crear una estructura típica con el comando:

```
linux:
$ ./odoo-bin scaffold my_module
~/src/odoo-addons/

windows (seg mi instalación):
cd "C:\Program Files\Odoo
19.0.20251220"
.\python\python .\server\odoo-bin
scaffold coches
.\server\extra-addons
```



Una vez creado un addon, lo primero es 'Actualizar lista de aplicaciones' para que la encuentre.

Por defecto al entrar en Aplicaciones, se activa el filtro Aplicaciones. Según como hayais declarado el addon (en manifest):

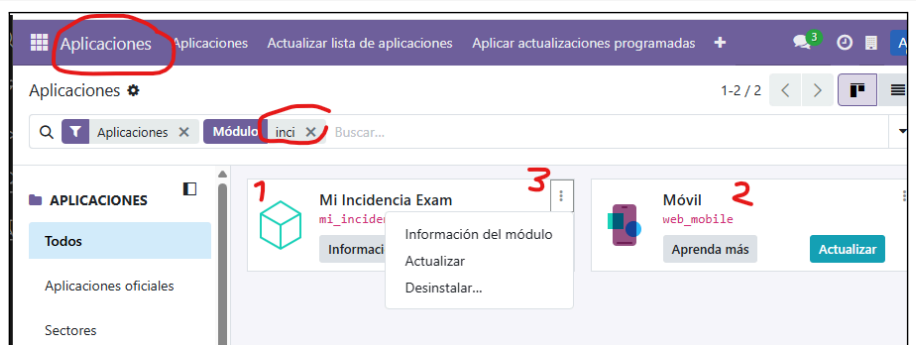
```
'installable': True,  
'application': True
```

lo encontrará o no. Otra cosa es que lo cargue (activar) a la primera.

Según desarrollamos el addon, lo vamos activando en Odoo desde Aplicaciones. En la figura se muestra cómo hemos buscado los addons/modulos en la ventana de Aplicaciones. Se muestra los módulos encontrados, de los que uno está instalado (1) y otro no (2). Para desinstalar (3)

Si **desinstalas** un módulo cuyos modelos crean tablas, estas se eliminan, mientras que si eliges **actualizar** y has eliminado un atributo, solo se elimina del interfaz de usuario pero no de la tabla. Si en data has especificado datos de prueba, depende de cómo los hayas especificado se añaden a los que había o no.

A veces he tenido que reiniciar el servicio Odoo y volver a 'Actualizar la lista de aplicaciones' para que 'limpie' bien la caché después de una descarga...



ALGO DE TEORÍA

- Un módulo/addon contiene:
 - modelos
 - vistas
 - acciones
 - menús
 - seguridad, datos, etc.

Un modelo representa datos.

Una vista muestra esos datos.

Una acción decide qué vista se abre.

Un menú llama a la acción.

- Odoo encaja bastante bien en el modelo MVC:

Modelo: models/

Vista: views/, report/* .xml

Controlador: controllers/

- La estructura de un addon es flexible y los ficheros pueden organizarse en distintas carpetas. Sin embargo, Odoo carga los ficheros XML en el orden indicado en el apartado *data* del fichero `__manifest__.py`.

Por ello, si un fichero referencia un elemento (por ejemplo, una acción o un report), ese elemento debe haberse cargado previamente.

Pej, He perdido mucho tiempo al diseñar un report: un botón en una vista que llama a la acción de impresión: la acción del report debe estar definida antes de que la vista la referencia.

- Ficheros más significativos:

__manifest__.py

Define el módulo: nombre, versión, dependencias, ficheros XML que se cargan, etc.
los campos más importantes son:

- **'depends':** donde se especifican los módulos de Odoo que tienen que estar cargados.
- **'data':** donde se especifican los ficheros y el orden en el que el analizador recorre los ficheros XML que especifican el add-on.

models/*.py

Define los modelos (<-> tablas), campos, lógica, métodos, estados, etc.

views/*.xml

Define la interfaz: vistas (form, list, kanban ...), acciones y menús.

security/ir.model.access.csv

Permisos básicos: quién puede leer, crear, editar o borrar en cada modelo. Un fallo típico es no ver el add-on aunque esté cargado por no especificar una regla

report/*.xml

Define informes QWeb PDF (plantilla y acción de impresión).

data/*.xml

Datos iniciales: estados, secuencias, configuraciones por defecto, etc.

De forma algo desordenada enumero algunas ideas/conceptos que creo que os pueden servir a la hora de desarrollar un add-on:

1. **Relación entre modelo y tablas (persistencia)**
2. **Vistas implícitas/explicitas.**
3. **Relación vista - acción.**
4. **Un módulo que modifica otro (herencia)**
5. **Modelo basado en una vista sql. (Report)**

1. Relación entre modelo y tablas

En el ejemplo se definen dos modelos (Coches y Propietarios) que implementan las tablas de la base de datos:
`x_apcoches_coches` y `x_apcoches_propietarios`.
(se cambia `'.'` por `'_'`)

Odoo crea automáticamente un campo `id` en cada modelo que sirve como identificador único de cada registro.

Como un propietario puede tener más de un coche, añadimos el campo relacional:

`coche_id`

=

```
class Coches(models.Model):  
    _name = 'x_apcoches.coches'  
    _description = 'ejemplo'  
  
    marca= fields.Char(string='Marca', required=True)  
    ...
```

```
class Propietarios(models.Model):  
    _name = 'x_apcoches.propietarios'  
    _description = 'ejemplo'  
  
    nombre= fields.Char(string='Nombre', required=True)
```

```
fields.Many2one('x_apcoches.propietarios',
string='Propietario')
```

En las vistas, Odoo se encarga de generar automáticamente el desplegable para seleccionar un propietario al dar de alta un coche, sin necesidad de programación adicional.

Cada modelo podría estar definido en su propio fichero (.py), o los dos en el mismo, pero siempre debajo de la carpeta /models.

```
apellidos= fields.Char(string='Apellidos', required=True)

coche_id= fields.Many2one('x_apcoches.coches', string='Coche')
...
```

Campos simples: almacenan datos básicos en la tabla del modelo. Ejemplos:

- fields.Char → texto
- fields.Text → texto largo
- fields.Integer → números enteros
- fields.Float → números decimales
- fields.Boolean → verdadero/falso
- fields.Date / fields.Datetime → fechas y horas

Campos de relación: enlazan registros de otros modelos:

- fields.Many2one('res.partner') → relación de uno a muchos (muchos registros de este modelo apuntan a uno del otro)
- fields.One2many('modelo.objetivo', 'campo_relacion') → relación de uno a muchos inversa (un registro "padre" tiene muchos hijos)
- fields.Many2many('modelo.objetivo') → relación de muchos a muchos entre modelos. Odoo crea automáticamente una tabla intermedia que guarda pares de IDs.

2. Vistas implícitas/explicitas.

Las **vistas implícitas** son generadas automáticamente si no definimos ninguna. Se crean basándose en el modelo y sus campos.

Solo sirven para interacción básica: el sistema muestra todos los campos visibles del modelo en un formulario o lista.

Las **vistas explícitas** las definimos nosotros en XML dentro de un módulo/addon. Tienen un ID y un nombre.

Pueden ser:

- Form (formulario)
- Tree / List (lista)
- Kanban
- Search (búsqueda)
- Pivot / Graph (informes)

El ejemplo define una vista de tipo form, con una cabecera en la que he metido un botón (que ejecuta la acción: *action_imprimir_registro* definida en otro sitio

```
<record id="view_mi_incidencia_form" model="ir.ui.view">
  <field name="name">mi.incidencia.form</field>
  <field name="model">x_mi_incidencia</field>
  <field name="arch" type="xml">
    <form string="Incidencia">
      <header>
        <button name="%(action_imprimir_registro)d"
          string="Imprimir informe"
          type="action"
          class="btn-primary"/>
      </header>
      <sheet>
        <group>
          <field name="name"/>
          <field name="request_date"/>
          <field name="user_id"/>
          <field name="state"/>
          <field name="priority"/>
          <field name="description"/>
        </group>
      </sheet>
    </form>
  </field>
</record>
```

3. Relacion vista - accion

Una **acción de ventana** (`ir.actions.act_window`) dice al sistema qué **modelo** queremos mostrar y cómo.

La acción tiene un campo **view_mode** que define qué vistas usar y en qué **orden**. En el ejemplo:

```
view_mode="kanban,form"
```

significa:

Primero intenta abrir la vista kanban. Si no está definida la crea (algo super sencillo, pero la crea). Si clicas en un elemento intenta abrir la vista formulario. Si no existe la crea (si puede).

```
<record id="action_mi_incidencia_kanban" model="ir.actions.act_window">
  <field name="name">Incidencias</field>
  <field name="res_model">x_mi_incidencia</field>
  <field name="view_mode">kanban,form</field>
</record>
```

Pregunta:

Puesto que una acción de ventana se define sobre un modelo ¿cómo se puede abrir una vista u otra?

Se puede definir el campo:

```
<field name="view_id" ref="mi_modelo_v1"/>
```

para explícitamente abrir la vista `mi_modelo_v`

OBS! Si no existe el campo `view_mode`, Odoo asume `tree,form`

4. Un módulo que modifica otro (herencia)

Imaginaros (muy, muy habitual) que ya hay un módulo en explotación, y que por lo que sea hay que añadirle un campo. No queremos modificar lo que hay (Hay datos en la tabla correspondiente!!!).

En este ejemplo tenemos el módulo `coches` y vamos a añadirle el campo `id_cliente`. El cliente tampoco existe, osea que necesitamos un mantenimiento de cliente ...

Las figuras muestran la creación del addon `coches_ext` que implementa lo anterior, es decir:

- Crear modelo para el cliente (el nombre del modelo se especifica en `_name` y es `x_coches.clientes`). No especificamos ninguna vista y dejamos que Odoo las gestione...
- Extender el modelo `x_coches.coches` con el campo `cliente_id`
- Extender la vista `form` `coches.form` para que muestre el campo `cliente_id`
- Enganchar cliente en el menú preexistente de coches..

Con estas pocas líneas de código tenemos un interface funcional:

Creamos la plantilla para el addon `coches_ext`:

```
\Program Files\Odoo 19.0.20251220>.python\python .\server\odoo-bin scaffold coches_ext_ .\server\extra-addons
```

En manifest, especificamos que depende del módulo `coches`:

```
'depends': ['base','coches'], #depende de coches
```

La clase `Clientes` que define el modelo `x_coches.clientes`:

```
class Clientes(models.Model):
    _name = 'x_coches.clientes' #El modulo de coches era x_coches.coche ...
    _description = 'x_coches.clientes'
    rec_name = 'nombre' # <- esto indica qué campo se muestra en Many2one

    nombre = fields.Char()
    dni = fields.Char()
```

OBS! Si no defines `_rec_name`, Odoo despliega algo así como `x_coches.cliente,1` al elegir un cliente para el coche.

La clase `CochesExt` que extiende el modelo `x_coches.coches`. (en lugar de `_name` que especifica el nuevo modelo, usa `_inherit` que especifica el modelo que extiende ...)

```
class CochesExt(models.Model):
    _inherit = 'x_coches.coches' #MI!! extiende

    cliente_id = fields.Many2one('x_coches.clientes', string="Cliente")
```

La acción para el mantenimiento de los clientes. Dejamos que Odoo cree las vistas a su aire ...

Coches Lista de coches Cientes

Nuevo Cientes ⚙

☐ Nombre

☐ Maria Gomez

y coches:

Coches Lista de coches Cientes

Nuevo Coches
x_coches.coches,1 ⚙ 📄 ✕

Marca	honda
Modelo	civic
Año de matriculación	2 feb 2020
Matricula	aaaaaa
Cliente	Maria Gomez

```
<record id="action_clientes" model="ir.actions.act_window">
  <field name="name">Clientes</field>
  <field name="res_model">x_coches.clientes</field>
  <field name="view_mode">list,form</field>
</record>
```

El menú para disparar la acción. El menú padre está definido en el addon coches ...

```
<menuitem id="menu_clientes"
  name="Clientes"
  parent="coches.menu_root"
  action="action_clientes"/>
```

Para añadir el campo cliente a la vista form de coche (coches.form), tengo que crear una vista que hereda de esta y añade el campo con xpath ... (de paso le digo que lo muestre en rojo)

```
<record id="coches_form_inherit" model="ir.ui.view">
  <field name="name">coches.form.inherit.cliente</field>
  <field name="model">x_coches.coches</field>
  <field name="inherit_id" ref="coches.form"/> <!-- el id de la vista original -->
  <field name="arch" type="xml">
    <xpath expr="//group" position="inside">
      <field name="cliente_id" style="color:red;"/>
    </xpath>
  </field>
</record>
```

Si quisiéramos incluir cliente_id en la vista lista haríamos lo mismo co con la vista original coches.list, pero cambiando el xpath:

```
<xpath expr="//field[@name='marca']" position="after">
  <field name="cliente_id"/>
</xpath>
```

Por último tenemos que editar el fichero de seguridad para dar permisos al modelo de clientes (x_coches.clientes):

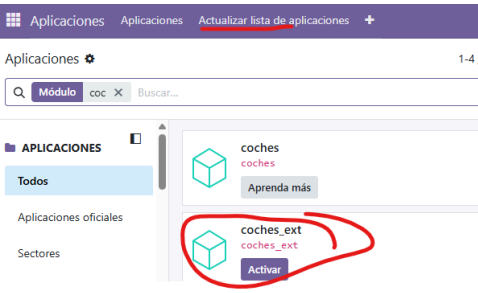
```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_coches_ext_clientes,Access Clientes,model_x_coches_clientes,,1,1,1,1
```

1-> el id que sea único

2-> una descripción

3-> esto tiene que coincidir con "model_" + el campo _name del modelo pero cambiando '.' por '_'

Listo. Acordaros de actualizar la lista de aplicaciones para poder activar el addon!

	
5. Modelo basado en una vista sql. (Report) Una función típica del dpto. de informática de una empresa es la gestión de distintos informes o extracción de datos de un ERP para distintos departamentos. En las figuras se muestra la creación del módulo/addon mis_reports que define dos modelos basados en vistas sql. La diferencia fundamental entre un modelo basado en datos y un modelo basado en una vista sql es la construcción: <pre>def init(self): tools.drop_view_if_exists(self.env.cr, 'el_report') self.env.cr.execute(""" CREATE OR REPLACE VIEW el_report AS SELECT c.id, c.campo1, c.campo2 FROM la_tabla c """)</pre> que especifica los datos. Los campos del modelo hacen referencia a los campos de la vista ... (menos id que no hay que poner): <pre>#id=fields.Integer(string='ID', readonly=True) marca = fields.Char(string='Marca') modelo = fields.Char(string='Modelo')</pre> También es necesario declarar vistas de tipo lista explícitas (aquí si queremos podemos establecer criterios de búsqueda y de agrupación)	Creo la plantilla para mis_reports: <pre>cd "C:\Program Files\Odoo 19.0.20251220" .\python\python .\server\odoo-bin scaffold mis_reports .\server\extra-addons</pre> <pre>C:\Windows\System32>cd "C:\Program Files\Odoo 19.0.20251220" C:\Program Files\Odoo 19.0.20251220>.\python\python .\server\odoo-bin scaffold mis_reports .\server\extra-addons C:\Program Files\Odoo 19.0.20251220>_</pre> Nombre del modelo, dependencias (1), seguridad (3) y acceso a su instalación (3): <pre>{ 'name': "mis_reports", 'summary': "Informes variados", 'description': ""Aquí voy metiendo informes sobre la marcha.", 'category': 'Uncategorized', 'version': '0.1', 'depends': ['base', 'x_coches'], 1 'data': ['security/ir.model.access.csv', 2 'views/views.xml', 'views/templates.xml',], 'installable': True, 3 'application': True, }</pre> 1 está mal. en lugar de x_coches es coches_ext. Depende del módulo coches_ext. El nombre del módulo está determinado por el nombre de la carpeta !!! (extra-addons/coches_ext que extiende coches y define clientes) Modelos basados en vistas sql:

```

class mis_reports(models.Model):
    _name = 'x_mis_reports.coches'
    _description = 'x_mis_reports.coches'
    _auto = False # no crear tabla fisica
    2 id = fields.Integer(string='ID', readonly=True)
    marca = fields.Char(string='Marca')
    modelo = fields.Char(string='Modelo')

    3 def init(self):
        tools.drop_view_if_exists(self.env.cr, 'x_mis_reports_coches') #segun _name = 'x_mis_reports.coches'
        self.env.cr.execute("""
            CREATE OR REPLACE VIEW x_mis_reports_coches AS
            SELECT c.id, c.marca, c.modelo
            FROM x_coches_coches c
            """)

class mis_reports(models.Model):
    _name = 'x_mis_reports.clientes'
    _description = 'x_mis_reports.clientes'
    _auto = False # no crear tabla fisica
    id = fields.Integer(string='ID', readonly=True)
    nombre = fields.Char(string='Nombre')
    dni = fields.Char(string='DNI')

    def init(self):
        tools.drop_view_if_exists(self.env.cr, 'x_mis_reports_clientes')
        self.env.cr.execute("""
            CREATE OR REPLACE VIEW x_mis_reports_clientes AS
            SELECT cl.id, cl.nombre, cl.dni
            FROM x_coches_clientes cl
            """)

```

1 no hay que crear tabla para los datos. (es una vista)

3 especifica el origen de los datos.

2 los campos del modelo hacen referencia a los campos de la vista.

OBS! tiene que haber un ID (recuerda que un modelo de datos lo crea de forma implícita ...)

2 está mal. en Odoo 19 NO hay que declararlo. Solo tiene que estar en la query

Acciones y menús. En views.xml:

```

<odoo>
  <data>
    <record id="action_report_coches" model="ir.actions.act_window">
      <field name="name">Informe Coches</field>
      <field name="res_model">x_mis_reports.coches</field>
      <field name="view_mode">list</field>
    </record>

    <!-- ACCIÓN CLIENTES -->
    <record id="action_report_clientes" model="ir.actions.act_window">
      <field name="name">Informe Clientes</field>
      <field name="res_model">x_mis_reports.clientes</field>
      <field name="view_mode">list</field>
    </record>

    <!-- MENÚ PRINCIPAL -->
    <menuitem id="menu_informes" name="Informes" sequence="10"/>

    <!-- SUBMENÚ COCHES -->
    <menuitem id="menu_report_coches" name="Coches" parent="menu_informes" action="action_report_coches" sequence="10"/>

    <!-- SUBMENÚ CLIENTES -->
    <menuitem id="menu_report_clientes" name="Clientes" parent="menu_informes" action="action_report_clientes" sequence="20"/>
  </data>
</odoo>

```

Editar /security/ir.model.access.csv para permitir a todos el acceso a los modelos:

```

id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_x_mis_reports_coches,access_x_mis_reports_coches,model_x_mis_reports_coches,,1,0,0,0
access_x_mis_reports_clientes,access_x_mis_reports_clientes,model_x_mis_reports_clientes,,1,0,0,0

```

Hay que declarar vistas de tipo list de forma explícita. (pensaba que bastaba con las vistas implícitas, pero no):
En views, al principio, antes de las acciones:

```
<!-- VISTA LISTA COCHES -->
<record id="report_coches_list" model="ir.ui.view">
  <field name="name">x_mis_reports.coches.list</field>
  <field name="model">x_mis_reports.coches</field>
  <field name="arch" type="xml">
    <list>
      <field name="marca"/>
      <field name="modelo"/>
    </list>
  </field>
</record>

<!-- VISTA LISTA CLIENTES -->
<record id="report_clientes_list" model="ir.ui.view">
  <field name="name">x_mis_reports.clientes.list</field>
  <field name="model">x_mis_reports.clientes</field>
  <field name="arch" type="xml">
    <list>
      <field name="nombre"/>
      <field name="dni"/>
    </list>
  </field>
</record>
```

Actualizar la lista de addons:

 Aplicaciones Aplicaciones Actualizar lista de aplicaciones Aplicar a

...